

Library- Subprogram- Package

Module Convenor: Dr.T. Nandha Kumar

Email: nandhakumaar.t@nottingham.edu.my

Department of Electrical & Electronic Engineering
The University of Nottingham Malaysia Campus



Topics to be covered

- Subprograms
 - Procedure
 - Function
- Library
- Package
- Subtype
- Type converters
- Shift Operators
- Attributes



Sub Programs

- VHDL allows the user to write sub-programs that can be called by the user and can be made available to a team.
- The statements in the subprogram are executed sequentially
- Two types of subprograms
 - Procedure
 - Function

Procedure

- A procedure consists of:
 - procedure declaration
 - can be within an **architecture** or a **package**
- a procedure call
 - in the architecture body as a concurrent statement
 - or in a process as a sequential statement
- The generic form of a procedure declaration is:
Procedure procedure_name [(parameter_declarations)] **is**
 { subprogram_declarative_item }
 begin
 { sequential_statement }
 end[procedure_name] ;



Procedure

Consider this simple example of a multiplier which is to be used for two separate multiplications such that:

$$\text{mult_a} = A \times B$$

$$\text{mult_b} = C \times D$$

Procedure

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 Entity two_mult is
7     Port (
8         A,B,C,D:in std_logic_vector (3 downto 0);
9         Mult_a, Mult_b:out std_logic_vector (7 downto 0));
10 end entity two_mult;
11
12 Architecture proc3 of two_mult is
13     --procedure declaration
14     Procedure mult (signal X,Y: in std_logic_vector (3 downto 0);
15         signal Mout: out std_logic_vector (7 downto 0))is
16         begin
17             --Mout <= ("0000"& X) * ("0000" & Y);--It will synthesis in ISE
18             Mout <= X * Y; --It will synthesis in ISE as well
19         end mult;
20     --end of procedure declaration
21 begin
22     mult (A,B,mult_a); --procedure call
23     mult (C,D,mult_b); --procedure call
24 end proc3;
25
26
```

Function

- A function is another type of subprogram and consists of:
 - function declaration
 - can be within an **architecture** or a **package**
- a function call
 - in the arch body as a concurrent statement
 - A function returns a result that can be used in an expression



Function

Many Functions already predefined within VHDL standard. For example:

rising_edge

If rising_edge (clock) **then**

type conversions i.e. --use range to limit the hardware



Function

General form of function declaration is similar to a procedure:

```
Function function_name [ ( parameter_declarations ) ] return  
type_name is  
    { subprogram_declarative_item }  
begin  
    { sequential_statement }  
end[ function_name ];
```

We must have at least one return statement inside a function

All functions return a single value

Function

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5 entity func_demo is
6     port(
7         signal A, B, OP: in std_logic;
8         signal RETURNED_VALUE: out std_logic
9     );
10
11 end func_demo;
12
13 architecture behave of func_demo is
14
15     --declaration region
16
17     function OPERATE(X, Y, OPERATION: std_logic) return std_logic is
18
19     begin
20         If (OPERATION = '1') then
21             return (X and Y);
22         else
23             return (X or Y);
24         end if;
25
26     end OPERATE;
27 begin
28
29     RETURNED_VALUE <= OPERATE(A, B, OP); --function call
30
31 end behave;
32
```

Function

```
ENTITY test_function IS    -- simple example of use of a function
    PORT(input            : IN INTEGER;
           output         : OUT INTEGER);
END test_function;

-- function with name "add_2" declared in architecture
-- function only visible to that architecture
ARCHITECTURE behavior OF test_function IS
    FUNCTION add_2 (value : IN integer)
        RETURN integer IS
            VARIABLE temp : integer; -- local variable
        BEGIN
            temp := value;
            temp := temp + 2;
            RETURN temp; -- value passed back via RETURN statement
        END add_2;
BEGIN
    output <= add_2(input); -- call of function, followed by arguments
END behavior;
```

Sub Programs....

■ function:

- ❑ executes in zero stimulation time.
- ❑ usually return a single value.
- ❑ parameters of mode IN.
- ❑ do not have wait statement.
- ❑ function call statement always concurrent.

■ procedure:

- ❑ may or may not execute in zero stimulation time depending upon whether it has wait statement or not.
- ❑ may or may not return a single value.
- ❑ parameters of in, out or inout.
- ❑ has wait statement.
- ❑ procedure call statement may be concurrent or sequential.

Library

- All complied components and packages are stored in the library
- It is globally available to all users

- Syntax

Library <library_name>;

Use <library_name>.<package_name>.ALL;

- Example

Library IEEE;

Use ieee.std_logic_1164.all;

Note : By default VHDL components are stored in the *WORK* library. *WORK* refers to a logical name and not the name of a directory

Packages

- Packages hold commonly used declarations such as:
 - constants
 - type declarations
 - subprograms (functions and procedures)
- Packages are placed:
 - in the same source file with architectureor
 - in a separate file and hence can be used by other projects
- We can write our own packages or....

Packages

Use the many predefined packages within VHDL i.e.

- `ieee.std_logic_1164`
- `ieee.numeric_std`
- `ieee.std_logic_unsigned` (synopsys) etc

This section will describe briefly how to write your own packages

- We have previously defined a function “operate” for local use. Let us now make this function available globally via the use of a package
- To do this, first wrap the function in our package as follows:

Packages

```
library ieee;  
use ieee.std_logic_1164.all;  
package my_pack is  
function MY_OPERATE(X, Y, OPERATION: std_logic) return std_logic;  
end my_pack;  
package body my_pack is  
function MY_OPERATE(X, Y, OPERATION: std_logic) return std_logic is  
begin  
    if (OPERATION = '1') then  
        return (X and Y);  
    else  
        return (X or Y);  
    end if;  
end MY_OPERATE;  
end my_pack;
```

Packages

- Notice that we have two parts to a package:
 - package declarations
 - package body
- File is compiled first so that it can be used in the current directory (WORK)
- A design unit is then written to use this function which is now globally available via our package 'my_pack':



Packages

```
library ieee;  
use ieee.std_logic_1164.all;  
library work;  
use work.my_pack.all;  
entity pack_demo is  
    port(  
        signal A, B, OP: in std_logic;  
        signal RETURNED_VALUE: out std_logic);  
end pack_demo;  
architecture behave of pack_demo is  
begin  
    RETURNED_VALUE <= MY_OPERATE(A, B, OP);  
end behave;
```

Packages

- In this case we only have declared a function in our package
- We can declare many other items in our package such as:
 - subprogram declaration
 - subprogram body
 - type and subtype declarations
 - constant declarations
 - shared variables
 - file and alias declarations
- By declaring in a package these can all be made globally available
- You should remember that these declarations can also be declared in an architecture body for local use

Subtype

- A '**subtype**' is a declaration of a data type that is a constrained version of an existing type
- For example:
 - **subtype** natural **is** integer **range**0 **to**2147483647 --i.e. (231-1)
 - **subtype** positive **is** integer **range**1 **to**2147483647
 - **subtype** word **is** std_logic_vector (3**downto**0);
- The first two are pre-defined in VHDL
- The last example is not pre-defined in VHDL
- It is called a user defined '**subtype**' and is very useful for aiding readability

Subtype in a Package

The package file would be:

```
library ieee;  
use ieee.std_logic_1164.all;  
package my_pack3 is  
  subtype word is std_logic_vector(3 downto 0);  
  function my_operate3(x, y: word; operation:  
    std_logic) return word;  
end my_pack3;  
package body my_pack3 is  
  function my_operate3(X, Y:word; operation :  
    std_logic) return word is  
  begin  
    if (operation = '1')then  
      return (X and Y);  
    else  
      return (X or Y);  
    end if;  
end my_operate3;  
end my_pack3;
```

The design unit would be:

```
library ieee;  
use ieee.std_logic_1164.all;  
library work;  
use work.my_pack3.all;  
entity pack_demo is  
  port(  
    signal A, B:word;  
    signal OP: in std_logic;  
    signal RETURNED_VALUE: out word);  
end pack_demo;  
architecture behave of pack_demo is  
  begin  
    RETURNED_VALUE <= my_operate3(A, B,  
      OP);  
end behave;
```

Type conversion

- Used to convert one data type to another
- Assigning one signal to another should be done only if they are of the same data type.
- The packages `std_logic_1164`, `std_logic_unsigned` contains type conversion functions
 - `Std_logic` $\leftrightarrow$ `Bit`
 - `Std_logic_vector` $\leftrightarrow$ `Bit_vector`
 - `Std_ulogic` $\leftrightarrow$ `Bit`
 - `Std_ulogic_vector` $\leftrightarrow$ `Bit_vector`
 - `Std_logic` to integer vice versa
- A good design must use less converters.

Type conversion

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity typeconverter is
    Port ( a : in  STD_LOGIC_VECTOR (4 downto 0);
          b : in  STD_LOGIC_VECTOR (4 downto 0);
          c : in  STD_LOGIC_VECTOR (4 downto 0);
          q : out STD_LOGIC_VECTOR (4 downto 0));
end typeconverter;

architecture Behavioral of typeconverter is

begin

    q <= a when conv_integer(b) = 9 else c;
end Behavioral;
```

Shift Operators

- Used to shift the data
- Applied to the signal in the bit_vector
- Six types
 - sll → Shift left
 - srl → shift right
 - rol → roll over left
 - ror → roll over right
 - sla → shift left and keep value right
 - sra → shift right and keep value left

Shift Operators

- Architecture *roll* of *roll_entity* is
begin
 a <= "01101";
 q1 <= a sll 1; -- q1 = "11010"
 q2 <= a srl 3; --q2 = "00001"
 q3 <= a rol 2; --q3 = "10101"
 q4 <=a ror 1; -- q4 = "10110"
 q5 <= a sla 2; --q5 = "10111"
 q6 <= a sra 1; --q6 = "00110"
end *roll*;



Attributes

- Signals can have predefined attributes associated with them and examples are
 - Function
 - Value
 - Type
 - Range
- Also possible to have user defined attributes

Function attributes

- Predefined functions
- Returns information about the behavior of signals
 - s'EVENT
 - returns true if an event occurred
 - change in value
 - s'ACTIVE
 - returns true if signal s is active
 - new value assigned to signal s (may be same value)
 - s'LAST_EVENT
 - returns elapsed time since last event
 - s'LAST_ACTIVE
 - s'LAST_VALUE
 - returns value of s before the last event

Function attribute Example

- Check for setup time violation on d input changing

```
CONSTANT setup_time : TIME := 12 ns; -- TIME is predefined
```

```
PROCESS (clk)
```

```
BEGIN
```

```
    IF clk'EVENT AND clk = '1' THEN
```

```
        ASSERT(d'LAST_EVENT >= setup_time)
```

```
            REPORT "setup violation"
```

```
            SEVERITY error;
```

```
    END IF;
```

- Assert statement checks that the input d has not had an event during the setup time.
- If time returned is less than setup time - assertion will fail

Range Attributes

- Only for constrained array types
- Returns range of specified type
- Two range attributes
 - a'RANGE
 - a'REVERSE RANGE

```
TYPE address_bus IS ARRAY (63 DOWNT0 0) OF std_logic;
```

```
SIGNAL cpu_address : address_bus; -- declare array
```

```
FOR j IN cpu_address'RANGE LOOP -- 63 DOWNT0 0
```

```
FOR j IN cpu_address'REVERSE_RANGE LOOP -- 0 TO 63
```



END of Lecture 7